
Hệ thống tiền xử lý văn bản tiếng Việt với cấu trúc Trie và thuật toán Aho-Corasick

Phan Gia Lộc^{1,*}, Đặng Ngọc Sang^{1,*}

¹Trường Đại học Khoa học tự nhiên

*Đóng góp ngang nhau

Tóm tắt nội dung

Chắc hẳn ai từng tiếp xúc với máy tính, đều từng thao tác qua phím tắt **Ctrl + F** để tìm kiếm một từ khóa trong một văn bản. Dưới góc nhìn thuật toán, có lẽ không ít người sẽ thắc mắc “*tại sao máy tính có thể xử lý và tìm kiếm từ khóa nhanh đến vậy?*”. Bài báo cáo đồ án môn học Cấu trúc dữ liệu và giải thuật này sẽ giúp giải đáp thắc mắc trên. Cụ thể, chúng tôi sẽ trình bày quá trình đánh giá hiệu năng của một hệ thống tiền xử lý văn bản tiếng Việt bằng bốn phương pháp tiếp cận khác nhau, từ đó kết hợp với các kiến thức nâng cao, chuyên sâu hơn về mặt phân tích thuật toán để hướng đến mục tiêu là xác thực tính ưu việt về mặt lý thuyết lẫn thực tế của cấu trúc Trie và thuật toán Aho-Corasick so với các phương pháp truyền thống.

Ngày: Tháng 7, năm 2026

Liên hệ: 2512000638@student.hcmus.edu.vn và 2512000706@student.hcmus.edu.vn

Mã nguồn: https://github.com/LeatuyrBertyk/CSC10004-Aho_Corasick

1 Bối cảnh, mục đích và phạm vi đồ án

1.1 Bối cảnh thực tế

Trong kỷ nguyên của Trí tuệ nhân tạo (AI) và Khoa học dữ liệu, hệ thống cần phải thu thập hàng chục triệu tài liệu văn bản thô từ Internet mỗi ngày. Để các Mô hình ngôn ngữ lớn (LLMs) có thể học được những tri thức chuẩn mực, bước đầu tiên và mang tính cốt lõi nhất là quy trình Tiền xử lý. Yêu cầu là cần phải tiến hành làm sạch dữ liệu bằng cách che khuất hàng chục ngàn từ khóa rác thuộc về một Từ điển nhiễu, bao gồm: **từ dừng**, các cụm từ vô nghĩa về mặt thông tin, ví dụ như “tuy nhiên”, “mặc dù là”; và **từ khóa rác**, các cụm từ quảng cáo rác, ví dụ như “quang cao shopee”, “san sale gia re”.

Có rất nhiều cách tiếp cận để xử lý quy trình che khuất dữ liệu nhiễu trên. Trong khóa học, chúng tôi đã được giới thiệu về thuật toán KMP (Knuth-Morris-Pratt), dù KMP hoạt động cực kỳ hiệu quả nhưng nó chỉ giải quyết được bài toán tìm kiếm đơn mẫu, tức tìm một mẫu hay từ khóa trong một văn bản. Khi đối mặt với hàng chục ngàn mẫu cần tìm, nếu chạy KMP lặp đi lặp lại cho mỗi đoạn văn bản, hệ thống sẽ sụp đổ về mặt thời gian thực thi. Để giải quyết bài toán tìm kiếm đa mẫu, sự tiến hóa về mặt tối ưu thuật toán đã được thể hiện bốn phương pháp tiếp cận: **Tìm kiếm vét cạn**, phương pháp tiếp cận vét cạn ngây thơ nhất, xử lý từng mẫu một cách hoàn toàn độc lập; **Trie tiêu chuẩn**, sử dụng Trie (cây tiền tố) để nhóm các từ khóa có chung tiền tố, sau đó duyệt qua văn bản và tìm kiếm dọc theo các nhánh; **Trie tối ưu hóa với Maximal Munch**, một chiến lược tham lam kết hợp với Trie để nhảy qua các từ đã được che khuất, giúp loại bỏ các vòng lặp

thừa; và **Thuật toán Aho-Corasick**, sự kết hợp hoàn hảo giữa cấu trúc và khái niệm Automaton của KMP. Nó cho phép quét qua văn bản với tốc độ tuyến tính tĩnh mà không bao giờ phải quay lui con trỏ.

1.2 Mục đích của báo cáo

Đồ án đưa ra nhằm giải thích về mặt thực nghiệm lý do tại sao cấu trúc Trie kết hợp với thuật toán Aho-Corasick lại giữ vị trí độc tôn trong kỹ thuật tìm kiếm và xử lý dữ liệu; bằng cách yêu cầu tiến hành xây dựng từ đầu một hệ thống cấu trúc hoàn chỉnh, xây dựng các thuật toán từ đơn giản đến nâng cao.

Thông qua các phương pháp đo lường hiệu năng, báo cáo sẽ cung cấp số liệu thực tế khi so sánh bốn phương pháp tiếp cận trong hai thực nghiệm là mở rộng kích thước văn bản, giữ nguyên độ lớn từ điển và mở rộng kích thước từ điển từ khóa rác, giữ nguyên độ lớn văn bản. Từ những dữ liệu thu thập được, báo cáo hướng tới việc phân tích sâu sắc ba chủ đề nâng cao:

- Nút thất quay lui trong các cách tiếp cận thông thường.
- Tác động của khả năng mở rộng từ điển.
- Chiến lược thuật toán Maximal Munch so với Khớp toàn bộ.

1.3 Phạm vi và yêu cầu kỹ thuật

Để đảm bảo mô hình phản ánh chính xác cơ chế hoạt động của phần cứng và hệ điều hành, hệ thống thực nghiệm được xây dựng tuân thủ các ràng buộc kỹ thuật sau:

Quản lý dữ liệu và lấy mẫu. Dữ liệu văn bản đầu vào phải được trích xuất từ tệp ngữ liệu `clean_corpus.txt` thông qua việc phân tách bằng khoảng trắng hoặc ký tự xuống dòng. Các từ sau đó được nối lại thành một chuỗi duy nhất cách nhau bởi một khoảng trắng. Đối với mỗi lượt thử nghiệm, hệ thống phải chọn ngẫu nhiên K mẫu từ tệp `noise_dict_50k.txt` thay vì chọn tuần tự từ đầu danh sách để tránh hiện tượng thiên lệch phân phối, lưu ý chỉ được phép sử dụng hàm `std::shuffle` cho bước xáo trộn này. Đồng thời, do quá trình che khuất thay thế trực tiếp ký tự thành `*` trên chuỗi, hệ thống bắt buộc phải tạo ra bốn bản sao chuỗi văn bản độc lập trước khi đưa vào hàm đo thời gian cho bốn thuật toán.

Cấu trúc dữ liệu lỗi. Trie phải được cài đặt bằng thiết kế `first_child/next_sibling` thông qua cấu trúc `TrieNode` nhằm tối ưu hóa bộ nhớ RAM. Do Trie được xây dựng bằng con trỏ nên để tránh tình trạng rò rỉ bộ nhớ, cần phải có hàm `freeTrie` để giải phóng và làm sạch toàn bộ bộ nhớ Heap sau khi sử dụng. Đồng thời, tuyệt đối nghiêm cấm sử dụng mảng động trong hàm thực thi tìm kiếm bằng thuật toán Aho-Corasick, thay vào đó, phải sử dụng mảng cấp phát tĩnh để tránh nút thắt cổ chai do hệ điều hành cấp phát lại vùng nhớ.

Thiết kế các thuật toán tìm kiếm. Hệ thống không được phép sử dụng thư viện hay bất kỳ hàm tìm kiếm tích hợp sẵn nào. Quá trình tìm kiếm phân biệt chữ hoa, chữ thường, tính cả ký tự khoảng trắng, và phải thay thế toàn bộ cụm từ tìm được bằng dấu `*`. Ngoài ra, còn có một số đặc thù cho bốn thuật toán, như đối với thuật toán Trie Tiêu chuẩn, phải có cơ chế quay lui khi việc so khớp thất bại; Trie Maximal Munch cần triển khai cơ chế tham lam, cho phép con trỏ nhảy qua toàn bộ phần văn bản vừa được che khuất thành công; và Aho-Corasick phải tận dụng con trỏ liên kết lỗi `fail_link` để quét liên tục, kết quả so khớp được gom vào mảng tĩnh và sắp xếp lại, không dùng các hàm có sẵn mà phải tự cài đặt lại hàm sắp xếp theo ưu tiên.

2 Đánh giá hiệu năng thực nghiệm

2.1 Môi trường thực thi và phương pháp đo lường

Để đảm bảo tính minh bạch của các kết quả thực nghiệm, toàn bộ quá trình biên dịch và đo lường được thực hiện trên một máy tính duy nhất là máy tính cá nhân của chúng tôi với mẫu máy Dell Inspiron 5458; vi xử lý Intel(R) Core(TM) i5-5250U hoạt động ở xung nhịp cơ bản 1.60GHz; bộ nhớ chính (RAM) có tổng dung lượng 12GB RAM chuẩn DDR3, tốc độ 1600 MT/s, được thiết lập chạy ở chế độ kênh đôi với hai thanh nhớ SODIMM (8GB của Samsung và 4GB của Micron); bộ nhớ lưu trữ là ổ cứng SSD ADATA SU650 với tổng

dung lượng khoảng 480GB; và hệ điều hành là Ubuntu 25.10 (x86-64), sử dụng lõi Linux Kernel phiên bản 6.17.0-35-generic.

Yêu cầu của quy trình đánh giá hiệu năng là chính xác đến mức độ nano giây, nhưng trên các hệ điều hành hiện đại thường xuyên bị nhiễu loạn bởi các yếu tố ngầm như các tiến trình chạy ngầm, và cơ chế đệm trang của hệ điều hành (OS Page Cache). Nhằm giảm thiểu tối đa các sai số vật lý này và thu thập được tốc độ thuần túy của thuật toán, chiến lược đo lường được thiết kế qua ba bước:

Khởi động. Tiến hành khởi chạy tệp thực thi lần đầu tiên sau khi tệp tin cơ sở dữ liệu nhị phân được tạo. Kết quả đo lường ở lần chạy này bị loại bỏ hoàn toàn do độ trễ khổng lồ từ việc đọc dữ liệu từ SSD lên RAM. Thao tác này nhằm ép hệ điều hành nạp sẵn các dữ liệu cần thiết vào vùng OS Page Cache.

Lấy mẫu. Khi dữ liệu đã nằm trên bộ nhớ đệm, hệ thống tiếp tục chạy tự động thao tác đánh giá 10 lần đọc lặp. Trong mỗi lần chạy, thuật toán phải thực hiện chính xác 100 truy vấn cho hai thực nghiệm.

Thống nhất số liệu. Đối với thời gian thực thi, báo cáo áp dụng phương pháp trích xuất giá trị thấp nhất trong 10 lần đo. Giá trị nhỏ nhất đại diện cho khoảng khắc vi xử lý ít bị hệ điều hành can thiệp nhất, phản ánh trung thực nhất năng lực giới hạn của từng thuật toán.

2.2 Tổng hợp kết quả quá trình thực nghiệm

Mục tiêu của phần thực nghiệm này là đánh giá và so sánh mức độ hiệu quả về mặt thời gian thực thi (tính bằng giây) của bốn thuật toán so khớp chuỗi: tìm kiếm vét cạn (Naive Search), cây tiền tố tiêu chuẩn (Standard Trie), cây tiền tố tối ưu hóa bằng Maximal Munch, và thuật toán Aho-Corasick. Thực nghiệm được thiết kế dưới hai thực nghiệm đối lập nhằm kiểm chứng độ phức tạp lý thuyết của từng thuật toán với dữ liệu thực tế: cố định quy mô từ điển ở mức $K = 10.000$ mẫu, tăng dần độ dài của văn bản đầu vào N từ 10.000 lên 10.000.000 từ để đánh giá khả năng duy trì thông lượng quét văn bản; và cố định độ dài văn bản ở mức $N = 1.000.000$ từ, tăng dần số lượng từ khóa trong từ điển K từ 1.000 lên 30.000 mẫu để kiểm định sự phụ thuộc của thời gian thực thi vào quy mô tập mẫu tìm kiếm.

Bảng 1 Kết quả thực nghiệm 1, cố định $K = 10,000$, tăng kích cỡ N .

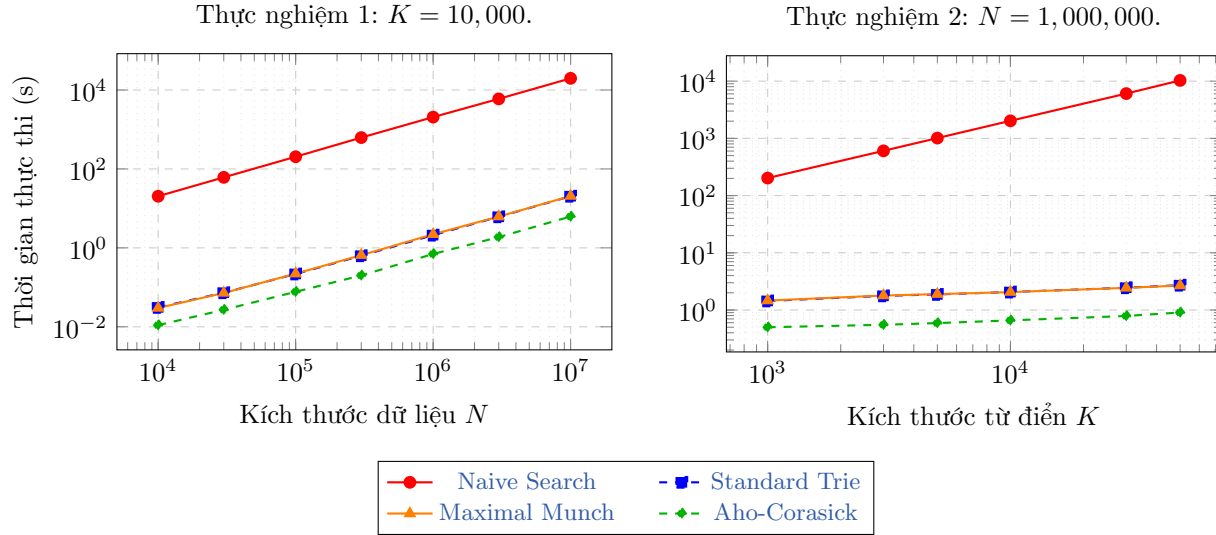
Kích thước N	Naive Search (s)	Standard Trie (s)	Maximal Munch (s)	Aho-Corasick (s)
10,000	20.36	0.031	0.030	0.011
30,000	61.24	0.073	0.072	0.027
100,000	203.87	0.218	0.222	0.077
300,000	623.08	0.631	0.653	0.200
1,000,000	2,059.06	2.101	2.209	0.700
3,000,000	5,963.37	6.128	6.205	1.874
10,000,000	19,758.71	20.380	20.588	6.232

Bảng 2 Kết quả thực nghiệm 2, cố định $N = 1,000,000$, tăng kích cỡ K .

Số từ khóa K	Naive Search (s)	Standard Trie (s)	Maximal Munch (s)	Aho-Corasick (s)
1,000	202.38	1.440	1.464	0.500
3,000	605.44	1.769	1.792	0.552
5,000	1,011.41	1.874	1.895	0.591
10,000	2,026.58	2.061	2.057	0.659
30,000	6,059.40	2.453	2.440	0.787
50,000	10,299.27	2.715	2.682	0.905

Hai bảng số liệu đã cung cấp chi tiết về các thông số kỹ thuật, nhưng để phân tích sâu hơn về xu hướng biến thiên của hệ thống khi khối lượng dữ liệu ngày càng lớn, các kết quả thực nghiệm cần được trực quan hóa.

Do dải dữ liệu thử nghiệm N và K trải dài trên một biên độ rất lớn, việc sử dụng hệ tọa độ tuyến tính thông thường sẽ làm không thể hiện rõ sự khác biệt ở các mốc nhỏ. Do đó, cả hai trục tọa độ đều được áp dụng tỷ lệ logarit cơ số 10, nhằm trực quan hóa sự khác biệt trên.



Hình 1 So sánh hiệu năng cho thực nghiệm 1 và 2.

Hai biểu đồ trên là minh chứng rõ nét cho sự phân hóa về hiệu năng giữa phương pháp tiếp cận vét cạn và các phương pháp ứng dụng cấu trúc dữ liệu cây hoặc Automaton. Cụ thể, ta có thể rút ra những nhận định sơ bộ sau:

Thứ nhất, sự sụp đổ của thuật toán tìm kiếm vét cạn trong bài toán tìm kiếm đa mẫu. Nhìn vào cả hai bảng 1 và 2, thuật toán luôn cho thấy thời gian thực thi tệ nhất và tăng trưởng theo cấp số nhân khi dữ liệu phình to.

Thứ hai, bước nhảy vọt của cấu trúc Trie và sự tương đồng của chiến lược Maximal Munch. So với thuật toán tìm kiếm vét cạn, cấu trúc Trie đã mang lại một bước nhảy vọt về hiệu năng. Ở bảng 1, Trie chỉ mất 20,380 giây so với hàng vạn giây của vét cạn. Tuy nhiên, giữa Trie tiêu chuẩn và Maximal Munch, thời gian thực thi không có sự chênh lệch đáng kể, thậm chí Maximal Munch đôi lúc chậm hơn một vài mili giây, ví dụ 20,588 giây so với 20,380 giây.

Thứ ba, sự thống trị tuyệt đối của Aho-Corasick. Thuật toán Aho-Corasick thể hiện một hiệu năng vượt trội và gần như bỏ xa tất cả các thuật toán còn lại. Ở mốc dữ liệu lớn nhất của bảng 1, Aho-Corasick chỉ mất 6,232 giây, nhanh hơn gấp 3 lần so với cấu trúc Trie và gấp hàng nghìn lần so với vét cạn.

Nhìn chung, thực nghiệm đã chứng minh rõ Aho-Corasick là lời giải chuẩn mực và hoàn hảo nhất cho bài toán tìm kiếm và lọc đa mẫu trên các ngữ liệu lớn, vừa bảo toàn được tốc độ xử lý tuyến tính tĩnh, vừa duy trì được tính ổn định trước sự bùng nổ của quy mô từ điển. *Lưu ý: từ phần sau của bài viết, khái niệm goto chỉ các con trỏ liên kết trong thiết kế cấu trúc TrieNode sử dụng mô hình first_child/next_sibling mà hệ thống yêu cầu.*

3 Phân tích độ phức tạp tổng quát của bốn thuật toán

Trong bài toán tìm kiếm và che khuất đa mẫu, việc thấu hiểu bản chất toán học và cơ chế vận hành của các cấu trúc dữ liệu là chìa khóa để giải thích sự chênh lệch hiệu năng khổng lồ giữa các phương pháp. Dưới đây là phần chứng minh độ phức tạp lý thuyết, đối chiếu với thực nghiệm và phân tích chuyên sâu về giới hạn của cấu trúc cây tiền tố.

Để thuận tiện, ta ký hiệu N là chiều dài của chuỗi văn bản đầu vào, K là số lượng từ khóa trong từ điển, M là chiều dài tối đa của một từ khóa, và Z là tổng số lần các từ khóa xuất hiện trong văn bản.

Tìm kiếm vét cạn. Cơ chế của thuật toán là duyệt qua từng từ khóa một cách độc lập, với mỗi từ khóa, nó quét qua toàn bộ chiều dài văn bản từ trái sang phải, và tại mỗi vị trí, nó so khớp từng ký tự của từ khóa với ký tự tương ứng trên văn bản. Vòng lặp ngoài cùng chạy K lần cho K từ khóa, vòng lặp giữa chạy N lần dọc theo chiều dài văn bản, tại mỗi vị trí trong văn bản, vòng lặp trong cùng tốn tối đa M phép so sánh để xác nhận một kết quả khớp. Do đó, trong trường hợp tồi tệ nhất, tổng số phép toán phải thực hiện là tỷ lệ thuận với tích của ba hằng số này. Đáng chú ý, như bài báo của Elvan Owen đã chỉ ra [2], phương pháp này lặp lại thao tác xử lý cho từng mẫu trong tập hợp K mẫu một cách cực kỳ dư thừa. Do vậy thuật toán có độ phức tạp $\mathcal{O}(K \times N \times M)$.

Trie tiêu chuẩn. Cơ chế của thuật toán là đưa toàn bộ K từ khóa vào một Trie, giúp gộp chung các tiền tố giống nhau. Khi duyệt văn bản, thuật toán bắt đầu từ gốc cây và đi xuống các nhánh tương ứng với từng ký tự. Nếu sự so khớp thất bại giữa chừng, con trỏ duyệt trên văn bản phải quay lui về vị trí bắt đầu cộng thêm 1, và lặp lại quá trình tìm kiếm từ gốc cây. Mặc dù Trie loại bỏ được hệ số K do không phải duyệt lại cho mỗi từ khóa, nhưng tại mỗi vị trí i trên văn bản có N vị trí, con trỏ có thể tiến sâu vào cây tối đa M bước trước khi phát hiện sai lệch và buộc phải quay lui. Hệ quả là một ký tự trên văn bản có thể bị đưa vào kiểm tra lặp đi lặp lại nhiều lần. Do vậy, độ phức tạp khi khởi tạo Trie là $\mathcal{O}(K \times M)$ và khi tìm kiếm là $\mathcal{O}(N \times M)$.

Trie tối ưu hóa bằng Maximal Munch. Cơ chế của thuật toán là vận hành trên cùng một nền tảng với Trie tiêu chuẩn, nhưng áp dụng thêm chiến lược tham lam. Khi thuật toán tìm thấy một từ khóa dài nhất tại vị trí i , nó thực hiện che khuất và cho phép con trỏ văn bản nhảy cóc qua toàn bộ chiều dài của từ khóa vừa khớp. Việc nhảy cóc giúp loại bỏ các bước lặp dư thừa trên những phần văn bản đã được che khuất. Tuy nhiên, sự tối ưu này chỉ phát huy tác dụng khi tìm thấy kết quả. Trong trường hợp tồi tệ nhất khi văn bản không chứa bất kỳ từ khóa nào nhưng liên tục gây ra các kết quả khớp một phần, thuật toán vẫn phải quay lui như Trie tiêu chuẩn. Do vậy độ phức tạp của thuật toán khi khởi tạo vẫn là $\mathcal{O}(K \times M)$, khi tìm kiếm trong trường hợp tốt nhất là $\mathcal{O}(N)$, và trong trường hợp tồi tệ nhất là $\mathcal{O}(N \times M)$.

Thuật toán Aho-Corasick. Cơ chế của thuật toán là kết hợp cấu trúc Trie với khái niệm Automaton trạng thái hữu hạn bằng cách bổ sung các Liên kết lỗi (Failure Links) và Liên kết đầu ra (Output/Dictionary Links). Khi việc so khớp trên một nhánh thất bại, cấu trúc tự động chuyển trạng thái dọc theo liên kết lỗi đến hậu tố hợp lệ dài nhất tiếp theo mà không cần lùi con trỏ văn bản. Theo công trình nguyên bản của A.V. Aho và M.J. Corasick (1975) [1], trong mỗi chu kỳ hoạt động, máy thực hiện 0 hoặc nhiều lần chuyển trạng thái lỗi, theo sau bởi đúng một lần chuyển trạng thái tiếp `goto transition`. Mỗi hàm `goto` sẽ tăng độ sâu của trạng thái hiện tại lên tối đa 1 mức, nghĩa là độ sâu chỉ có thể tăng tối đa N lần cho chuỗi chiều dài N . Mỗi hàm `fail` lại đưa trạng thái di chuyển gần về phía gốc hơn. Do độ sâu không thể là số âm, tổng số lần thực hiện hàm `fail` trong toàn bộ quá trình quét không bao giờ vượt quá tổng số lần hàm `goto` tăng độ sâu. Do đó, tổng số lần chuyển trạng thái tối đa chỉ là $2N$, bất kể số lượng từ khóa hay độ dài của chúng. Cùng với chi phí để thu thập Z kết quả là $\mathcal{O}(Z)$, độ phức tạp của thuật toán là $\mathcal{O}(N + Z)$.

4 Nút thắt quay lui và sự bảo toàn đặc tính tuyến tính của thuật toán Aho-Corasick

Dựa trên cơ sở lý thuyết về độ phức tạp đã được thiết lập ở phần 3, phần này sẽ đi sâu vào việc đối chiếu dữ liệu thực nghiệm nhằm làm rõ ranh giới hiệu năng giữa Trie tiêu chuẩn và thuật toán Aho-Corasick. Trọng tâm của phân tích là tác động của cơ chế quay lui và cách khái niệm Automaton triệt tiêu hoàn toàn nút thắt cổ chai này.

4.1 Minh chứng thực nghiệm về đặc tính tuyến tính tuyệt đối của Aho-Corasick

Khảo sát dữ liệu trong thực nghiệm 1, ta có thể rút ra một nhận xét mang tính quyết định rằng thuật toán Aho-Corasick bộc lộ đặc tính tuyến tính một cách tuyệt đối, thể hiện qua việc khi kích thước văn bản N tăng lên theo một tỷ lệ nhất định, thời gian thực thi của nó cũng tăng lên theo chính xác tỷ lệ đó. Sự tương quan một đối một này được minh chứng cực kỳ sắc nét qua các mốc nội suy từ bảng 1: khi N tăng gấp 3 lần từ 100.000 lên 300.000 từ, thời gian vận hành của Aho-Corasick tăng từ 0,077 giây lên 0,200 giây, tức tỷ lệ tăng khoảng 2,6 lần; đặc biệt chuẩn xác khi N tiếp tục tăng gấp 3 lần từ mốc 1.000.000 lên 3.000.000 từ, thời gian chạy tăng từ 0,700 giây lên đúng 1,874 giây, tức tỷ lệ tăng khoảng 2,67 lần; còn ở mốc kiểm thử cực đại, khi

N tăng xấp xỉ 3,33 lần từ 3.000.000 lên 10.000.000 từ, thời gian thực thi tăng tương ứng từ 1,874 giây lên 6,232 giây, tức tỷ lệ tăng khoảng 3,32 lần.

Hiện tượng tăng trưởng thời gian khớp hoàn hảo với sự gia tăng của khối lượng dữ liệu đầu vào đã xác nhận rằng cơ chế Automaton của Aho-Corasick quét qua văn bản với số lượng phép toán không đổi trên mỗi ký tự, hoàn toàn không bị ảnh hưởng bởi các vòng lặp ẩn hay sự gia tăng của số lượng mẫu kề nhau.

4.2 Bản chất của nút thất bại trong cấu trúc Trie tiêu chuẩn

Mặc dù Trie tiêu chuẩn mang lại tốc độ vượt trội so với tìm kiếm vét cạn (tốn 20,380 giây so với 19.758,71 giây của vét cạn ở mức 10.000.000 từ), thuật toán này vẫn luôn chậm hơn Aho-Corasick từ 3 đến 3,5 lần trên mọi thang đo thực nghiệm. Sự suy giảm hiệu năng này có nguồn gốc sâu xa từ một lỗ hổng kiến trúc nút thất bại.

Trong quá trình duyệt văn bản bằng Trie, hệ thống duy trì một con trỏ trạng thái di chuyển dọc theo các nhánh của cây, tuy nhiên, cấu trúc Trie cơ bản mang tính chất phi bộ nhớ đối với các chuỗi con đã khớp một phần. Cụ thể, khi thuật toán duyệt được một tiền tố dài nhưng gặp thất bại ở ký tự cuối cùng, nó sẽ vứt bỏ toàn bộ những thông tin đã so khớp thành công trước đó. Con trỏ duyệt trên văn bản bắt buộc phải lùi lại về vị trí ngay sau ký tự bắt đầu của lần khớp đó, và quá trình tìm kiếm lại được khởi động từ gốc của Trie. Hậu quả của cơ chế này là thời gian quét không còn duy trì được tính tuyến tính trên thực tế, tùy thuộc vào sự phân bố của dữ liệu, một ký tự trên văn bản có thể bị hệ thống đọc, tải vào bộ đệm và so sánh lặp đi lặp lại nhiều lần. Trong khi đó, thuật toán Aho-Corasick giải quyết triệt để bài toán này bằng cách đưa vào khái niệm liên kết lỗi (Failure Link). Theo thiết kế của Aho và Corasick (1975) [1], khi xảy ra sự cô không khớp, thay vì quay lui con trỏ văn bản, trạng thái của Automaton tự động dịch chuyển dọc theo liên kết lỗi đến hậu tố hợp lệ dài nhất tiếp theo, điều này đảm bảo rằng con trỏ văn bản chỉ có một nhiệm vụ duy nhất là tiến về phía trước, giữ vững thông lượng quét tối đa.

4.3 Thiết kế tập dữ liệu đặc biệt để minh chứng rõ lỗ hổng

Để làm nổi bật sự vô hiệu quả của nút thất bại quay lui, chúng ta có thể thiết kế một tập dữ liệu đối kháng nhằm đẩy cấu trúc Trie vào trường hợp tồi tệ nhất, ép thuật toán này phải kích hoạt cơ chế lùi con trỏ liên tục. Cấu trúc tập dữ liệu đối kháng được xây dựng: Từ điển nhiều với $K = 1$ chỉ chứa một mẫu duy nhất có độ dài $M = 10$, cấu tạo bởi 9 ký tự ‘a’ và kết thúc bằng ký tự ‘b’, cụ thể: [‘aaaaaaaaab’]; ngữ liệu đầu vào $N = 10.000.000$ gồm một chuỗi văn bản khổng lồ chỉ chứa toàn ký tự ‘a’, hoàn toàn không có ký tự ‘b’, cụ thể: “aaaaaaaaaaaaaaaaaaaaa...”.

Sự sụp đổ của Trie tiêu chuẩn được thể hiện qua quy trình: ở bước 1, Trie đọc 9 ký tự ‘a’ đầu tiên của văn bản, đi rất sâu xuống nhánh của cây, đến ký tự thứ 10, văn bản đưa vào là ‘a’ nhưng Trie đang mong đợi ‘b’. Quá trình so khớp thất bại; đến bước 2, Trie tiêu chuẩn phá hủy toàn bộ trạng thái hiện tại, lùi con trỏ văn bản lại vị trí tiếp theo và xem như hủy bỏ 9 bước tiến trước đó; tiếp tục bước 3, thuật toán lại mù quáng lặp lại quá trình như trên, tiến 9 bước qua các chữ ‘a’, thất bại ở chữ ‘a’ thứ 10, và lại quay lui về vị trí tiếp theo.

Với văn bản dài 10.000.000 ký tự, con trỏ của Trie tiêu chuẩn sẽ bị ép phải dịch chuyển qua lại xấp xỉ $10.000.000 \times 10 = 100.000.000$ lần. Hệ thống bị vướng vào một vòng lặp cục bộ khiến chi phí quét bị đội lên theo cấp số nhân với độ dài mẫu. Ngược lại, nếu đưa tập dữ liệu này qua Aho-Corasick, thông qua đồ thị trạng thái, các liên kết lỗi sẽ trở các trạng thái ‘a’ chéo về nhau. Thuật toán sẽ chạy băng qua toàn bộ 10.000.000 ký tự văn bản chỉ trong đúng một lần quét tiến mà không cần phải lùi con trỏ dù chỉ một bước, minh chứng cho sức mạnh tối thượng của kiến trúc Automaton trong xử lý ngôn ngữ tự nhiên.

5 Tác động của khả năng mở rộng từ điển và thuộc tính độc lập của thuật toán Aho-Corasick

Khi triển khai các hệ thống lọc dữ liệu văn bản trong thực tế, quy mô của từ điển nhiều bao gồm từ dừng, từ khóa rác hiếm khi giữ nguyên mà luôn có xu hướng phình to theo thời gian. Khảo sát thực nghiệm 2 đóng vai

trò then chốt trong việc đánh giá tính bền vững và khả năng mở rộng của các kiến trúc thuật toán.

5.1 Sự sụp đổ của tìm kiếm vết cạn và ưu thế của cấu trúc Trie

Nhìn vào dữ liệu thực nghiệm ở bảng 2, hình thái tăng trưởng thời gian thực thi của phương pháp tìm kiếm vết cạn vẽ ra một đường thẳng tuyến tính dốc đứng, hoàn toàn tỷ lệ thuận với số lượng từ khóa K . Cụ thể, khi kích thước từ điển K tăng gấp 50 lần (từ 1.000 lên 50.000 mẫu), thời gian thực thi của vết cạn cũng bùng nổ theo tỷ lệ tương ứng, tăng xấp xỉ 50 lần từ 202,38 giây lên mức khủng khiếp 10299,27 giây. Sự sụp đổ này bắt nguồn từ bản chất lặp rồi rác của thuật toán, mỗi khi một mẫu mới được thêm vào từ điển, hệ thống phải thực hiện thêm một vòng quét độc lập từ đầu đến cuối trên toàn bộ văn bản N . Ngược lại, các thuật toán dựa trên cấu trúc Trie thể hiện sức chịu đựng tuyệt vời trước sự phình to của dữ liệu. Cùng với mức tăng gấp 50 lần của K , thời gian của Trie tiêu chuẩn chỉ nhích chậm chạp từ 1,440 giây lên 2,714 giây. Bằng cách hợp nhất các tiền tố chung của từ khóa vào cùng một nhánh cây, cấu trúc Trie đã triệt tiêu được thao tác quét lại văn bản nhiều lần đối với các từ khóa có hình thái bắt đầu giống nhau. Tuy nhiên, sự tối ưu này của Trie vẫn chưa đạt đến mức độ hoàn hảo do nó vẫn chịu ảnh hưởng gián tiếp từ sự phân nhánh ngày càng dày đặc của cây khi K lớn, kéo theo chi phí cho quá trình quay lui cũng tăng nhẹ.

5.2 Bản chất toán học của Aho-Corasick, sự độc lập tuyệt đối với quy mô từ điển

Điểm đáng chú ý nhất của bảng 2 chính là đồ thị thời gian gần như đi ngang của thuật toán Aho-Corasick. Khi K tăng gấp 30 lần (từ 1.000 lên 50.000), thời gian vận hành của Automaton chỉ tăng một lượng cực kỳ vi ngót từ 0,500 giây lên 0,905 giây. Hiện tượng thực nghiệm này minh chứng cho thuộc tính toán học vĩ đại nhất của Aho-Corasick là khả năng tìm kiếm trong thời gian tuyến tính gần như hoàn toàn độc lập với quy mô của từ điển. Theo công trình nền tảng của A.V. Aho và M.J. Corasick (1975) [1], số lượng chuyển đổi trạng thái mà trạng thái hữu hạn thực hiện khi xử lý văn bản là một hằng số đối với mỗi ký tự đầu vào, bất kể có bao nhiêu từ khóa tồn tại trong từ điển. Các tác giả đã chứng minh toán học một cách chặt chẽ rằng máy sẽ thực hiện tổng cộng không vượt quá $2N$ lần chuyển trạng thái trong toàn bộ quá trình quét văn bản độ dài N .

Khi K tăng lên, cấu trúc nội bộ của Automaton, bao gồm các liên kết `goto` và `fail_link` trở nên chằng chịt và phức tạp hơn trong giai đoạn tiền xử lý. Thế nhưng, một khi cỗ máy đã được xây dựng xong, lúc tiến hành quét trên đoạn văn bản, con trỏ trạng thái chỉ đơn thuần lướt trên một mạng lưới các đồ thị đã được tính toán sẵn. Việc có 1.000 hay 50.000 điểm đích trong từ điển hoàn toàn không làm thay đổi số bước tiến của con trỏ trên văn bản.

5.3 Phân tích chuyên sâu về nút thất bại xử lý ở mốc dữ liệu cực đại

Có một chi tiết thực nghiệm mà ta cần phải phân tích rạch ròi là mặc dù thời gian quét lỗi của Automaton là tuyến tính, tại sao thời gian thực tế của Aho-Corasick không cố định vĩnh viễn ở mức 0,500 giây mà vẫn xuất hiện xu hướng tăng biên lên 0,905 giây ở mốc $K = 50.000$? Sự gia tăng thời gian này hoàn toàn không xuất phát từ cơ chế so khớp chuỗi của Automaton, mà nó đến từ khâu thu thập và hậu xử lý kết quả, cụ thể:

Sự bùng nổ biến Z . Theo lý thuyết chuẩn, độ phức tạp thực sự của Aho-Corasick khi tính cả việc xuất kết quả là $\mathcal{O}(N + Z)$, trong đó Z là số lượng kết quả trùng khớp được phát hiện. Khi tập từ điển K phình to từ 1.000 lên 50.000, xác suất để các chuỗi con trong đoạn văn bản vô tình khớp với một từ khóa trong từ điển tăng lên đáng kể. Do đó, Z bùng nổ tỷ lệ thuận với K .

Chi phí ghi nhận và sắp xếp. Trong yêu cầu kỹ thuật, thuật toán bắt buộc phải thu thập tất cả Z kết quả trùng lặp vào một mảng động `Match* matches`, và ngay sau đó gọi hàm `sortMatches` có độ phức tạp $\mathcal{O}(Z \log Z)$ để sắp xếp lại mảng này trước khi đề ký tự `*`.

Do đó, khi K đạt những mốc vô cùng lớn, Automaton vẫn chỉ mất chính xác một khoảng thời gian không đổi vài mili-giây để quét qua văn bản, nhưng toàn bộ sự gia tăng thời gian được ghi nhận trên bảng 2 chính là phần chi phí phụ trợ bắt buộc để máy tính cấp phát bộ nhớ, thu thập một mảng Z kết quả ngày càng khổng lồ, chạy thuật toán sắp xếp trên mảng đó, và thực thi vòng lặp thay thế `*` trên chuỗi gốc. Từ đó, ta có nhận xét rằng thuật toán Aho-Corasick đã tách rời thành công độ phức tạp của bài toán so khớp văn bản khỏi kích

thước của tập từ điển, và mọi sự hao tổn thời gian phát sinh ở các mốc dữ liệu cực đại hoàn toàn chỉ thuộc về cách thức quản lý mảng kết quả của hệ thống máy tính.

6 Chiến lược thuật toán Maximal Munch so với Khớp toàn bộ và bài toán toàn vẹn dữ liệu

Nếu như hai phần trước tập trung vào việc phân tích thông lượng và độ phức tạp thời gian, thì phần này lại đặt ra một lăng kính đánh giá hoàn toàn khác, đó là tính đúng đắn của ngữ nghĩa và sự toàn vẹn của dữ liệu. Sự đánh đổi giữa tốc độ và độ chính xác được bộc lộ rõ nét qua việc đối chiếu cơ chế tham lam của Maximal Munch với nguyên lý Khớp toàn bộ của Automaton trên thuật toán Aho-Corasick.

6.1 Cơ chế tham lam và sự tương đồng thực nghiệm

Theo định nghĩa từ yêu cầu kỹ thuật, Trie tối ưu hóa với Maximal Munch vận hành dựa trên một chiến lược tham lam, ngay khi thuật toán xác định được một từ khóa dài nhất khớp thành công (giả sử có độ dài L), hệ thống sẽ tiến hành thay thế trực tiếp bằng các ký tự $*$ và lập tức thực hiện hành vi nhảy cóc bằng cách cộng thêm L vào biến lặp i của vòng lặp duyệt văn bản ngoài cùng.

Về mặt lý thuyết, động thái nhảy cóc này mang lại một ưu thế toán học khổng lồ, nó triệt tiêu hoàn toàn các vòng lặp dư thừa bên trong phần văn bản đã được che khuất. Thuật toán không bao giờ phải lùi con trỏ hay tiến hành quét lại những ký tự đã chắc chắn bị loại bỏ, giúp cấu trúc Trie xấp xỉ đạt được thời gian tuyến tính trong các trường hợp văn bản chứa mật độ từ khóa rác dày đặc.

Tuy nhiên, khi đối chiếu với dữ liệu thực nghiệm tại bảng 1 và bảng 2, ta lại thấy một hiện tượng thú vị, thời gian thực thi của Maximal Munch luôn bám sát, thậm chí đôi khi chậm hơn vài mili-giây so với Trie tiêu chuẩn, ví dụ ở mốc $N = 10.000.000$, Maximal Munch tốn 20,588 giây trong khi Trie tiêu chuẩn tốn 20,380 giây. Sự tương đồng này phản ánh đặc thù của tập ngữ liệu kiểm thử, nếu văn bản không chứa quá nhiều chuỗi từ khóa lồng ghép kéo dài, số lượng vòng lặp được tiết kiệm nhờ cơ chế nhảy cóc sẽ không đủ để bù đắp lại các chi phí phụ trợ của việc kiểm tra điều kiện nhảy. Dù vậy, triết lý thiết kế của Maximal Munch vẫn là một cột mốc quan trọng trong việc tối ưu hóa cấu trúc Trie.

6.2 Bài toán xung đột che khuất và sự sai lệch thống kê

Mặc dù chiến lược Maximal Munch và các thuật toán còn lại đều hướng đến việc làm sạch văn bản, nhưng quá trình thay thế trực tiếp lại dẫn đến một hệ lụy nghiêm trọng về mặt dữ liệu, đó là sự mất mát toàn phần đối với các từ khóa lồng ghép. Phân tích sâu hơn, ngay cả thao tác thu thập toàn bộ kết quả của Aho-Corasick thông qua mảng `Match*` cũng bộc lộ những khiếm khuyết logic khi áp dụng vào bài toán che khuất. Xét một tập dữ liệu đối kháng ngữ nghĩa với chuỗi văn bản đầu vào là `“shoppeeii”` và từ điển $K = \{\text{“shop”}, \text{“shoppee”}, \text{“eeii”}\}$.

Sự mất mát dữ liệu của Maximal Munch và Trie tiêu chuẩn. Khi quét đến vị trí $i = 0$, Maximal Munch (với ưu tiên chuỗi dài nhất) sẽ khớp thành công từ `“shoppee”` và lập tức che khuất nó thành `*****ii`. Ngay sau đó, con trỏ i nhảy qua 7 ký tự này và quét tiếp từ chữ `‘i’`. Hậu quả là từ khóa `“eeii”` bị bỏ qua hoàn toàn và không bao giờ được phát hiện. Tương tự với Trie tiêu chuẩn hay tìm kiếm vét cạn, một khi chuỗi gốc đã bị ghi đè bằng dấu $*$, các từ khóa phía sau vĩnh viễn bị tước đoạt môi trường so khớp, dẫn đến hiện tượng che thiếu trầm trọng.

Nghịch lý thống kê của Aho-Corasick với `sortMatches`. Thuật toán Aho-Corasick xử lý bài toán phát hiện ưu việt hơn bằng cách thu thập tất cả kết quả so khớp vào một mảng `Match*`. Đối với Automaton, nó sẽ bắt được trọn vẹn 3 kết quả, là `(“shop”, 0, 3)`, `(“shoppee”, 0, 6)`, và `(“eeii”, 5, 8)`. Thế nhưng, sự sai lệch lại nảy sinh ở khâu hậu xử lý. Theo yêu cầu, hàm `sortMatches` phải sắp xếp mảng ưu tiên `start` tăng dần và `end` giảm dần, tạo ra trình tự duyệt `[(0, 6), (0, 3), (5, 8)]`. Khi lặp qua mảng này để che khuất, chuỗi `(0, 6)` được xử lý trước, biến văn bản thành `*****ii`; sau đó thuật toán bỏ qua `(0, 3)` do nằm gọn trong vùng đã che; đến chuỗi `(5, 8)`, thuật toán tiếp tục thực thi lệnh đè ký tự $*$ lên i từ 5 đến 8. Mặc dù về mặt thị giác, kết quả cuối cùng vẫn là một chuỗi toàn dấu $*$, nhưng về mặt kiến trúc phần mềm, hệ thống đã thực thi một thao tác che khuất chông chéo. Ký tự $*$ bị ghi đè lên chính ký tự $*$ đã tồn tại ở i bằng 5 và 6.

Hậu quả là hệ thống thống kê báo cáo có 3 từ rác được tìm thấy và xử lý, nhưng thực tế diện tích vùng nhớ rác chỉ tương đương với 2 cụm từ. Sự sai lệch logic này có thể làm sụp đổ các mô hình học máy nếu chúng dựa vào ma trận tần suất đếm từ khóa để huấn luyện.

Như vậy, cả chiến lược Maximal Munch lẫn phương pháp dùng `sortMatches` của Aho-Corasick đều vấp phải những rào cản thuật toán. Maximal Munch hy sinh tính toàn vẹn dữ liệu để lấy tốc độ duyệt nhảy cóc, trong khi Aho-Corasick bảo toàn được sự toàn vẹn trong khâu phát hiện, nhưng thao tác thay thế tại chỗ trực tiếp lại phá vỡ tính nhất quán của dữ liệu thống kê.

6.3 Ứng dụng thực tiễn: lựa chọn chiến lược trong xử lý ngôn ngữ tự nhiên

Sự khác biệt về triết lý giữa việc bỏ qua các kết quả lồng ghép và ghi nhận không sót một kết quả nào quyết định trực tiếp đến phạm vi áp dụng của hai kiến trúc này trong Khoa học dữ liệu thực tiễn:

Bài toán áp dụng Maximal Munch. Điển hình là hệ thống tách từ tiếng Việt, bởi đặc thù của Tiếng Việt là các từ đơn có thể ghép lại với nhau tạo thành từ ghép mang ngữ nghĩa hoàn toàn mới. Giả sử ta có chuỗi văn bản “`học sinh học sinh học`” và từ điển bao gồm “`học`”, “`sinh`”, “`học sinh`”, “`sinh học`”, việc áp dụng chiến lược Maximal Munch với ưu tiên khớp cụm từ dài nhất và nhảy cóc, là một cơ chế hoàn hảo để tách chuỗi này thành tập các cụm chuẩn xác [“`học sinh`”, “`học`”, “`sinh học`”]. Nếu áp dụng cơ chế khớp toàn bộ của Aho-Corasick ở đây, hệ thống sẽ trả về một ma trận kết quả bùng nổ, đan chéo nhau một cách hỗn loạn, khiến các mô hình ngôn ngữ không thể xác định được ranh giới từ hợp lệ để thực hiện các bước nhúng từ.

Bài toán áp dụng khớp toàn bộ tương tự Aho-Corasick. Điển hình là hai hệ thống nhận dạng trình tự y sinh và tra cứu thư mục. Trong các hệ thống tra cứu thư mục y khoa đồ sộ, thuật toán Aho-Corasick đã được chứng minh là tăng tốc độ tìm kiếm từ 5 đến 10 lần. Các tác giả A.V. Aho và M.J. Corasick (1975) [1] đã chỉ ra rằng hệ thống Automaton này cực kỳ thiết yếu trong việc nhận diện tất cả các sự xuất hiện của chuỗi. Trong ngành tin sinh học, khi tìm kiếm các mẫu DNA hỏng trên một dải gene khổng lồ, việc sử dụng chiến lược nhảy cóc có thể dẫn đến thảm họa y tế. Ví dụ, nếu chuỗi “`eeii`” chứa thông tin về một dạng đột biến gene, nhưng nó lại bị ẩn sâu bên trong chuỗi gen bình thường “`shoppeeii`”, Maximal Munch sẽ bỏ qua hoàn toàn đột biến này, chỉ có hệ thống Automaton của Aho-Corasick với năng lực quét tuyến tính mới có thể ghi nhận đầy đủ mọi chuỗi DNA lồng ghép, đảm bảo tính toàn vẹn thông tin một cách tuyệt đối cho các nghiên cứu di truyền học.

7 Kết luận

Với vai trò là một sinh viên ngành công nghệ thông tin, việc tìm hiểu các cấu trúc dữ liệu có tính ứng dụng cao là một điều rất quan trọng. Trong khóa học lý thuyết Cấu trúc dữ liệu và giải thuật, chúng tôi đã được giới thiệu qua thuật toán KMP và nhắc sơ về thuật toán Aho-Corasick nhưng vẫn chưa thực sự tìm hiểu sâu về nó. Và đồ án này đã giúp nhóm hiện thực hóa cấu trúc dữ liệu Trie và thuật toán Aho-Corasick trên bằng ngôn ngữ C++, đồng thời tạo cơ hội để nhóm có thể tự tay chạy kiểm nghiệm và tìm hiểu nhiều kiến thức chuyên sâu hơn về phân tích thuật toán, rất hữu ích cho công việc và nghiên cứu học thuật. Ngoài ra, qua việc đối chiếu kết quả thực nghiệm với các nền tảng lý thuyết cơ sở dữ liệu, chúng tôi đã được làm sáng tỏ lý do vì sao Trie và Aho-Corasick được xem như là tiêu chuẩn cho các hệ thống tiền xử lý ngôn ngữ.

Thực hiện báo cáo này còn giúp nhóm làm quen với kỹ năng nghiên cứu học thuật thông qua việc cài đặt mã nguồn để hoàn thiện ý tưởng, tích cực đọc các bài báo khoa học để có cái nhìn chuyên sâu hơn, từ đó vận dụng các kỹ năng như phân tích và tổng hợp dữ liệu, trình bày văn bản học thuật và kỹ năng tư duy khoa học để đưa ra bài báo cáo này một cách hoàn thiện và chỉnh chu nhất. Điều khó khăn nhất mà chúng tôi gặp phải trong khi thực hiện đồ án này là thời gian thực thi, do trong mỗi trường hợp, mỗi thuật toán cần chạy 100 lần để lấy kết quả trung bình nên máy tính cần chạy trong thời gian dài, đặc biệt thuật toán tìm kiếm vét cạn gây mất rất nhiều thời gian. Nhóm đã phải để máy chạy xuyên nhiều đêm để có thể ghi nhận kết quả đầy đủ và chính xác nhất, dù vậy, những kết quả và dữ liệu đáng giá đã được thu nhận nhờ sự kiên trì và quyết tâm của các thành viên.

Tài liệu

- [1] Alfred V. Aho and Margaret J. Corasick. Efficient string matching: An aid to bibliographic search. Communications of the ACM, 18(6):333–340, June 1975.
- [2] Elvan Owen. Aho–corasick algorithm in pattern matching, 2015. Makalah IF2211 Strategi Algoritma – Sem. II Tahun 2014/2015, Institut Teknologi Bandung.